Embedded Systems (M233)
The Four Midterms, Solved

Midterm 2022–2023 (bâtiment intelligent et santé), 2023–2024 (transport et mobilité), 2024–2025 (industrie, agriculture et énergie) and Midterm 2 (21 October 2026): every exercise answered in full, every sketch run in the simulator.

Papers by Pr. Aissam Berrahou (UM6P).
Solutions written and verified independently (the official answer keys were not available).

All 18 sketches in `kit/midterms/` compile for the ATmega328P and pass `python3 midterms/verify_midterms.py`.

Summary: What the four papers have in common

Same format every year: 2 h 30, 8–10 independent exercises, 20 points, Arduino UNO R3 (ATmega328P, 16 MHz, 5 V), *all timing non-blocking with* `millis()`, code in readable commented C/C++. The exercise *types* recur with new scenarios:

1. **Knowledge** (2 pts): QCM / true-false / fill-in: embedded-system types (Stand-Alone, Real-Time, Networked, Mobile), ATmega328P facts (8-bit AVR RISC, 16 MHz, Flash 32 KB, SRAM 2 KB, EEPROM 1 KB, Serial buffer 64 bytes, 40 mA max / 20 mA recommended per pin), `setup`/`loop`/hidden `main`, `static`, `#define` vs `const`, `Serial.read()` returns −1, PWM pins 3 5 6 9 10 11, type sizes.
2. **Platform / system-type case study** (1.5–2 pts): classify devices and justify (UNO vs ESP32 vs STM32 vs Raspberry Pi).
3. **Wiring** (2–2.5 pts): a Tinkercad breadboard with numbered wires: match wires to pins from the constants; recognise external pull-down, external pull-up, internal pull-up; LED resistor calculation $R = (5 - V_F)/I$ with E12 values; diagnose errors (no resistor, reversed LED, wrong pin, TX/RX conflict).
4. **Code reading** (2–2.5 pts): fill blanks (debounce, `INPUT_PULLUP`, `!led`, `millis()`), trace output (`static`, `sizeof`, `parseInt` and the stray `'\n'`, `=` vs `==`), find five bugs (`int` for `millis()`, floating input, missing `pinMode`, `if(...);`, `delay`).
5. **Labs recall** (1–1.5 pts): LED chain D13 → resistor → anode → cathode → GND; breadboard rows and rails; floating inputs in simulators; edge vs level detection; `#ifdef DEBUG`; parallel LED branches.
6. **FSM analysis** (2.5–3.5 pts): fill labels, write the transition table, trace states at given instants, find the safety flaw, propose the corrected machine.
7. **FSM design** (2.5–4.5 pts): boolean input expressions, full state diagram, complete non-blocking sketch (`enum class`, `switch`, `tEntree`, blinking by `millis()`), one justification question, test scenarios.

The appendix (§5) is the one-page sheet of every number and template the papers ask for.

Contents

1 Midterm 2022–2023: bâtiment intelligent et santé **3**

1.1 Exercice 1 : vrai ou faux, justifié (2 pts) . . . 3
1.2 Exercice 2 : associer concepts et définitions (1,5 pt) . . . 3
1.3 Exercice 3 : équipements d'un hôpital (2 pts) . . . 4
1.4 Exercice 4 : diagnostic du montage éclairage d'une chambre (2,5 pts) . . . 4
1.5 Exercice 5 : lampe à trois modes, blancs et trace (2,5 pts) . . . 5
1.6 Exercice 6 : retour sur les labs (1,5 pt) . . . 6
1.7 Exercice 7 : distributeur de gel hydroalcoolique (3,5 pts) . . . 6
1.8 Exercice 8 : appel infirmier (4,5 pts) . . . 7

2 Midterm 2023–2024: transport et mobilité **9**

2.1 Exercice 1 : QCM (2 pts) . . . 9
2.2 Exercice 2 : tableau des types (1,5 pt) . . . 9
2.3 Exercice 3 : ordonner et compléter (1,5 pt) . . . 10
2.4 Exercice 4 : calculs, éclairage d'une trottinette (2 pts) . . . 10
2.5 Exercice 5 : montage à compléter, clignotants de vélo (2,5 pts) . . . 10
2.6 Exercice 6 : débogage, passage à niveau (2 pts) . . . 11
2.7 Exercice 7 : trace d'exécution, afficheur de quai (1,5 pt) . . . 12
2.8 Exercice 8 : analyse de la FSM du passage à niveau (3 pts) . . . 12
2.9 Exercice 9 : feu tricolore avec bouton piéton et mode nuit (4 pts) . . . 14

3 Midterm 2024–2025: industrie, agriculture et énergie **15**

3.1 Exercice 1 : questions à réponse courte (2 pts) . . . 15
3.2 Exercice 2 : schéma de principe de la serre (1,5 pt) . . . 16
3.3 Exercice 3 : QCM (2 pts) . . . 16
3.4 Exercice 4 : montage à compléter, station de pompage (2,5 pts) . . . 16
3.5 Exercice 5 : structures, pointeurs et références (2 pts) . . . 17
3.6 Exercice 6 : bilan énergétique du nud capteur (1,5 pt) . . . 17
3.7 Exercice 7 : retour sur les labs (1,5 pt) . . . 17
3.8 Exercice 8 : de la table de transitions au code, remplissage de bouteilles (3 pts) . . . 18
3.9 Exercice 9 : conception de la FSM d'arrosage (4 pts) . . . 19

4 Midterm 2 (21 octobre 2026): industrie, automobile, surveillance **20**

4.1 Exercice 1 : QCM (2 pts) . . . 20

4.2 Exercice 2 : compléter les concepts (1,5 pt) . . . 20
4.3 Exercice 3 : choix d'une plateforme (1,5 pt) . . . 21
4.4 Exercice 4 : entrées/sorties d'une presse industrielle (2 pts) . . . 21
4.5 Exercice 5 : analyse de code (2,25 pts) . . . 22
4.6 Exercice 6 : montage à compléter, trois boutons, trois LEDs (2,5 pts) . . . 22
4.7 Exercice 7 : retour sur les labs (1 pt) . . . 23
4.8 Exercice 8 : analyse de la FSM de l'alarme (2,5 pts) . . . 23
4.9 Exercice 9 : alarme de magasin, montage et code (2,25 pts) . . . 25
4.10 Exercice 10 : rappel de ceinture de sécurité (2,5 pts) . . . 26

5 Appendix: the numbers and templates the papers keep asking 28

1 Midterm 2022–2023: bâtiment intelligent et santé

1.1 Exercice 1 : vrai ou faux, justifié (2 pts)

Réponse

1. **Faux.** Une Raspberry Pi est un *ordinateur monocarte* (SoC ARM, Linux, RAM externe) ; un microcontrôleur intègre CPU, Flash, SRAM et périphériques sur une puce et exécute un seul programme sans OS (ATmega328P).
2. **Vrai.** Portable, sur batterie, contraintes d'énergie et de taille : type Mobile.
3. **Faux.** L'EEPROM est *non volatile* : elle conserve ses 1 Ko (réglages) sans alimentation ; c'est la SRAM (2 Ko) qui perd les variables.
4. **Faux.** Le cur Arduino fournit un `main()` caché : `init(); setup(); for(;;) loop();`. Le programmeur écrit `setup()` et `loop()`.
5. **Vrai.** D0 = RX, D1 = TX de l'UART (USB / moniteur série) ; on évite de les utiliser comme E/S.
6. **Faux.** Avec `INPUT_PULLUP` la résistance interne tire la broche à 5 V : relâché = `HIGH`, appuyé (vers GND) = `LOW`.
7. **Vrai.** `static` donne une durée de vie globale à une variable à portée locale : initialisée une fois, conservée entre les appels.
8. **Vrai.** `Serial.read()` renvoie -1 quand le tampon de réception est vide (d'où le test `Serial.available() > 0`).

1.2 Exercice 2 : associer concepts et définitions (1,5 pt)

Réponse

A1 Flash → B4 (non volatile, programme compilé) ; A2 SRAM → B1 (volatile, variables) ; A3 EEPROM → B8 (non volatile, réglages modifiables) ; A4 pull-down → B6 (force à 0 V au repos) ; A5 anti-rebond → B2 ; A6 FSM → B5. Non utilisées : B3 (LoRa, protocole radio longue portée) et B7 (CAN, convertisseur analogique-numérique).

1.3 Exercice 3 : équipements d'un hôpital (2 pts)

Réponse

Équipement	Type	Contrainte principale	Si non respectée
a) Pompe à perfusion portable	Mobile	autonomie énergétique, taille/poids	la pompe s'arrête en déplacement : perfusion interrompue, surdosage ou sous-dosage
b) Moniteur cardiaque relié au poste	Networked	fiabilité et latence de la communication (données continues)	mesures perdues ou en retard : arythmie non vue par les infirmiers
c) Défibrillateur automatique	Real-Time	échéance temporelle stricte (choc synchronisé sur le cycle cardiaque)	choc hors fenêtre : fibrillation induite, danger vital
d) Autoclave sans connexion	Stand-Alone	fiabilité/autonomie du cycle (température, durée, sécurité pression)	cycle incomplet : instruments non stériles, infections

1.4 Exercice 4 : diagnostic du montage éclairage d'une chambre (2,5 pts)

Lecture du schéma (constantes : `BOUTON = 12`, `LAMPE = 13`, `VEILLEUSE = 11`) : fil noir GND → rail − ; fil orange D13 → colonne 10 = anode de la LED LAMPE (cathode col. 11 → rail − par un cavalier) ; fil vert D11 → col. 16 → résistance 220 Ω (col. 16–19) → col. 19 de la VEILLEUSE ; fil bleu : du bouton (col. 5) à la broche **TX 1**, l'autre patte du bouton (col. 3) au rail −.

Réponse

a) Les trois erreurs :

1. **LAMPE sans résistance série** : D13 attaque directement l'anode. Conséquence : courant limité seulement par la LED (> 40 mA), LED et/ou broche détruites. Correction : 220 Ω entre D13 et l'anode (ou entre cathode et GND).
2. **Bouton câblé sur TX 1 (D1) au lieu de D12**. Conséquence : `digitalRead(12)` lit toujours `HIGH` (pull-up interne, rien de branché) : le bouton n'a aucun effet ; de plus D1 est la ligne TX du `Serial.begin(9600)` : le bouton court-circuite les trames vers le PC. Correction : déplacer le fil bleu sur D12.
3. **VEILLEUSE montée à l'envers** : le + (anode, patte longue) est sur la colonne 20, reliée au rail GND, et la résistance arrive sur la cathode (col. 19). Conséquence : la LED est polarisée en inverse, elle ne s'allume jamais (pas de dégât : 5 V < tension inverse max). Correction : retourner la LED (anode côté résistance, cathode côté GND).

b) `pinMode(BOUTON, INPUT_PULLUP)` active la résistance de rappel interne ($\approx$ 20–50 kΩ) vers 5 V ; le bouton relie la broche à GND. Sans elle, broche relâchée = entrée *flottante* (valeur aléatoire).

c) Relâché : `HIGH` (tiré à 5 V par le pull-up). Appuyé : `LOW` (relié à GND).

1.5 Exercice 5 : lampe à trois modes, blancs et trace (2,5 pts)

Réponse

a) (1) `T_DEBOUNCE` ; (2) `LOW` (front descendant = appui, bouton vers GND) ; (3) `INPUT_PULLUP` ; (4) `Mode::BLINK` ; (5) `HIGH` ; (6) `!led`. **b)** L'appui est pris en compte après 50 ms de stabilité (1,05 s ; 2,05 s ; 4,05 s). En BLINK la LED part à `HIGH` et bascule toutes les 500 ms (2,55 s ; 3,05 s ; 3,55 s. . .).

Instant	0,5 s	1,5 s	2,2 s	2,7 s	4,5 s
Mode	OFF	ON	BLINK	BLINK	OFF
LED	éteinte	allumée	allumée	éteinte	éteinte

```cpp
// Midterm 2022-2023, Exercice 5 : lampe a trois modes (blancs completes)
const int BOUTON = 12, LED = 13;
const unsigned long T_BLINK = 500, T_DEBOUNCE = 50;
enum class Mode { OFF, ON, BLINK };
Mode mode = Mode::OFF;
bool led = LOW;
unsigned long tBlink = 0;
int lecturePrec = HIGH, etatStable = HIGH;
unsigned long tChange = 0;

bool appui() {                          // vrai une seule fois par appui
  int l = digitalRead(BOUTON);
  if (l != lecturePrec) { tChange = millis(); lecturePrec = l; }
  if (millis() - tChange > T_DEBOUNCE && l != etatStable) {   // (1) T_DEBOUNCE
    etatStable = l;
    return etatStable == LOW;                                 // (2) LOW : front descendant = appui
  }
  return false;
}

void setup() {
  pinMode(BOUTON, INPUT_PULLUP);                              // (3) INPUT_PULLUP (bouton vers GND)
  pinMode(LED, OUTPUT);
}

void loop() {
  if (appui()) {
    switch (mode) {
      case Mode::OFF:   mode = Mode::ON; break;
      case Mode::ON:    mode = Mode::BLINK;                   // (4) Mode::BLINK
                        led = HIGH; tBlink = millis(); break;
      case Mode::BLINK: mode = Mode::OFF; break;
    }
  }
  switch (mode) {
    case Mode::OFF: digitalWrite(LED, LOW); break;
    case Mode::ON:  digitalWrite(LED, HIGH); break;           // (5) HIGH
    case Mode::BLINK:
      if (millis() - tBlink >= T_BLINK) { tBlink = millis(); led = !led; }   // (6) !led
      digitalWrite(LED, led);
      break;
  }
}
```

In practice. Simulateur : appuis à 1, 2 et 4 s ; LED13 haute de 1,056 à 2,556 s, basse jusqu'à 3,056, haute jusqu'à 3,556, basse ensuite : la trace ci-dessus.

1.6 Exercice 6 : retour sur les labs (1,5 pt)

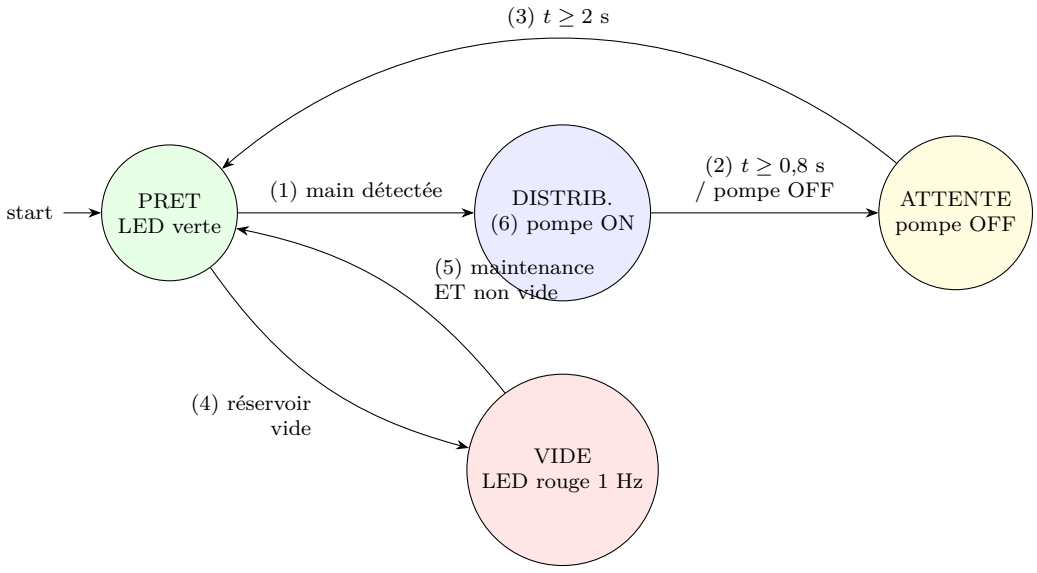
Réponse

1. D13 → résistance 220 Ω → anode (+, patte longue) → cathode → GND. LED à l'envers : polarisée en inverse, elle bloque, rien ne s'allume (aucune destruction à 5 V).
2. Les rails horizontaux (première rangée Vcc, dernière GND) donnent une convention lisible : on repère d'un coup d'il un fil d'alimentation mal placé ; vérifier avant la mise sous tension évite un court-circuit 5 V–GND (qui fait chuter la tension ou grille le régulateur) ou une LED sans résistance.
3. Le simulateur donne à une entrée non reliée une valeur définie (souvent 0), donc un bouton sans pull-up/pull-down marche en simulation. Sur la carte réelle la broche est flottante : elle capte le bruit, lit tantôt HIGH tantôt LOW, et le programme déclenche au hasard. Toujours fixer le niveau de repos (`INPUT_PULLUP` ou résistance externe).

1.7 Exercice 7 : distributeur de gel hydroalcoolique (3,5 pts)

Réponse

a) Étiquettes : (1) PRET → DISTRIB. : *main détectée* (D2 = LOW) ; (2) DISTRIB. → ATTENTE : $t \geq 0{,}8$ s / pompe OFF ; (3) ATTENTE → PRET : $t \geq 2$ s ; (4) PRET → VIDE : *réservoir vide* (D3 = LOW) ; (5) VIDE → PRET : *bouton maintenance appuyé ET réservoir non vide* (D4 = LOW et D3 = HIGH) ; (6) sortie de DISTRIB. : *pompe ON* (LED verte éteinte).



Réponse

b) ATTENTE impose une pause de 2 s pompe arrêtée après chaque dose : une seule dose par présentation de main, le temps de retirer la main. Sans lui, tant que la main reste devant le capteur la FSM enchaînerait PRET → DISTRIB. sans interruption : pompage continu, gaspillage, flaque.
c) Énumération et cases PRET / VIDE : voir le sketch (lignes 7 et 28–46) ; le clignotement 1 Hz utilise `tCligno` et `millis()`, jamais `delay`.
d) Non : dans DISTRIB. et ATTENTE le niveau n'est pas testé, la dose en cours se termine à vide (pompe qui tourne à sec). Amélioration : tester `reservoirVide()` aussi dans DISTRIB. (et

ATTENTE) pour passer immédiatement en VIDE, comme dans le sketch (ligne 33) ; variante : mémoriser le manque pour couper après la dose.

```
// Midterm 2022-2023, Exercice 7 : distributeur de gel (FSM complete, non bloquante)
const int CAPTEUR_MAIN = 2, NIVEAU_BAS = 3, BTN_MAINT = 4;
const int POMPE = 8, LED_VERTE = 9, LED_ROUGE = 10;
const unsigned long T_DOSE = 800, T_ATTENTE = 2000, T_CLIGNO = 500;   // 1 Hz = 500 ms ON / 500 ms OFF

enum class Etat { PRET, DISTRIB, ATTENTE, VIDE };
Etat etat = Etat::PRET;
unsigned long tEntree = 0, tCligno = 0;
bool ledRouge = false;

bool mainDetectee()   { return digitalRead(CAPTEUR_MAIN) == LOW; }
bool reservoirVide()  { return digitalRead(NIVEAU_BAS) == LOW; }
bool maintenance()    { return digitalRead(BTN_MAINT) == LOW; }
void changer(Etat e)  { etat = e; tEntree = millis(); }

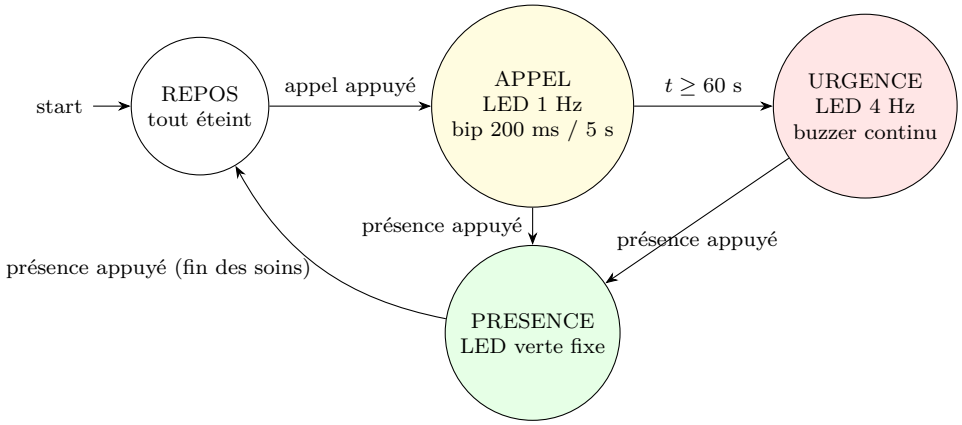
void setup() {
  pinMode(CAPTEUR_MAIN, INPUT);          // le capteur IR a sa propre sortie
  pinMode(NIVEAU_BAS, INPUT_PULLUP);
  pinMode(BTN_MAINT, INPUT_PULLUP);
  pinMode(POMPE, OUTPUT); pinMode(LED_VERTE, OUTPUT); pinMode(LED_ROUGE, OUTPUT);
}

void loop() {
  unsigned long now = millis();
  switch (etat) {
    case Etat::PRET:                                  // (demande : code du case PRET)
      digitalWrite(LED_VERTE, HIGH); digitalWrite(LED_ROUGE, LOW); digitalWrite(POMPE, LOW);
      if (reservoirVide())      changer(Etat::VIDE);     // (4)
      else if (mainDetectee())  changer(Etat::DISTRIB);  // (1)
      break;
    case Etat::DISTRIB:                               // (6) : pompe ON, LED verte eteinte
      digitalWrite(LED_VERTE, LOW); digitalWrite(POMPE, HIGH);
      if (reservoirVide())      changer(Etat::VIDE);     // amelioration d) : securite pendant la dose
      else if (now - tEntree >= T_DOSE) changer(Etat::ATTENTE);   // (2)
      break;
    case Etat::ATTENTE:
      digitalWrite(POMPE, LOW);
      if (now - tEntree >= T_ATTENTE) changer(Etat::PRET);        // (3)
      break;
    case Etat::VIDE:                                  // (demande : code du case VIDE)
      digitalWrite(POMPE, LOW); digitalWrite(LED_VERTE, LOW);
      if (now - tCligno >= T_CLIGNO) { tCligno = now; ledRouge = !ledRouge; digitalWrite(LED_ROUGE, ledRouge); }
      if (maintenance() && !reservoirVide()) changer(Etat::PRET);  // (5) : valide seulement si rempli
      break;
  }
}
```

1.8 Exercice 8 : appel infirmier (4,5 pts)

Réponse

a) Un bouton a quatre données : sa broche, son dernier niveau *stable*, sa dernière lecture *brute* et l'instant du dernier changement brut (pour les 50 ms d'anti-rebond). On les regroupe dans `struct Bouton` et on écrit `bool appui(Bouton &b)`. Par *référence* parce que la fonction doit *modifier* ces champs (mettre à jour `etatStable`, `lecturePrec`, `tChange`) et que ces modifications doivent survivre au retour : passée par valeur, la fonction travaillerait sur une copie, l'état d'anti-rebond serait perdu à chaque appel et aucun appui ne serait jamais détecté.



Réponse

c) Sketch complet ci-dessous. Points de barème à faire apparaître : `enum class`, `switch` par état, `tEntree` mémorisé à chaque changement, clignotements et bips calculés avec `millis()` (le bip de 200 ms toutes les 5 s est `(ecoule % 5000) < 200`), lecture des deux boutons *à chaque tour* avant le `switch`, aucune fonction bloquante.
d) URGENCE garantit qu'un appel non pris en charge finit par être *impossible à ignorer* (sirène continue, LED rapide) : un patient qui chute ou fait un malaise ne peut pas rappeler ; l'escalade automatique compense l'absence de réponse humaine. Améliorations : afficher le numéro de chambre au poste (liaison série / réseau) et journaliser les délais de réponse ; bouton d'appel à tirette au lit ; répétition de l'alerte au poste après un délai en PRESENCE trop long.

```
// Midterm 2022-2023, Exercice 8 : appel infirmier (sketch complet, non bloquant)
const int BTN_APPEL = 2, BTN_PRESENCE = 3;
const int LED_COULOIR = 8, LED_VERTE = 9, BUZZER = 10;
const unsigned long T_DEBOUNCE = 50, T_URGENCE = 60000UL;
const unsigned long T_BIP = 200, T_PERIODE_BIP = 5000, T_LED_1HZ = 500, T_LED_4HZ = 125;

enum class Etat { REPOS, APPEL, URGENCE, PRESENCE };   // declare avant toute fonction qui l'utilise
Etat etat = Etat::REPOS;
unsigned long tEntree = 0, tLed = 0;
bool led = false;

struct Bouton {                 // a) toutes les donnees d'un bouton : la fonction les modifie -> par reference
  byte pin;
  int etatStable;               // dernier niveau stable (HIGH au repos avec INPUT_PULLUP)
  int lecturePrec;              // derniere lecture brute
  unsigned long tChange;        // instant du dernier changement brut
};
Bouton appel    = {BTN_APPEL,    HIGH, HIGH, 0};
Bouton presence = {BTN_PRESENCE, HIGH, HIGH, 0};

bool appui(Bouton &b) {         // vrai une seule fois par appui (front descendant debounce)
  int l = digitalRead(b.pin);
  if (l != b.lecturePrec) { b.tChange = millis(); b.lecturePrec = l; }
  if (millis() - b.tChange > T_DEBOUNCE && l != b.etatStable) {
    b.etatStable = l;
    return l == LOW;
  }
  return false;
}

void changer(Etat e) { etat = e; tEntree = millis(); tLed = millis(); led = false; }

void setup() {
  pinMode(BTN_APPEL, INPUT_PULLUP); pinMode(BTN_PRESENCE, INPUT_PULLUP);
  pinMode(LED_COULOIR, OUTPUT); pinMode(LED_VERTE, OUTPUT); pinMode(BUZZER, OUTPUT);
}

```

```ino
void loop() {
  bool a = appui(appel), p = appui(presence);   // lire les deux boutons a chaque tour
  unsigned long now = millis(), ecoule = now - tEntree;
  switch (etat) {
    case Etat::REPOS:
      digitalWrite(LED_COULOIR, LOW); digitalWrite(LED_VERTE, LOW); digitalWrite(BUZZER, LOW);
      if (a) changer(Etat::APPEL);
      break;
    case Etat::APPEL:
      if (now - tLed >= T_LED_1HZ) { tLed = now; led = !led; digitalWrite(LED_COULOIR, led); }
      digitalWrite(BUZZER, (ecoule % T_PERIODE_BIP) < T_BIP);      // 200 ms toutes les 5 s
      if (p) changer(Etat::PRESENCE);
      else if (ecoule >= T_URGENCE) changer(Etat::URGENCE);
      break;
    case Etat::URGENCE:
      if (now - tLed >= T_LED_4HZ) { tLed = now; led = !led; digitalWrite(LED_COULOIR, led); }
      digitalWrite(BUZZER, HIGH);
      if (p) changer(Etat::PRESENCE);
      break;
    case Etat::PRESENCE:
      digitalWrite(LED_COULOIR, LOW); digitalWrite(BUZZER, LOW); digitalWrite(LED_VERTE, HIGH);
      if (p) changer(Etat::REPOS);                                  // fin des soins
      break;
  }
}
```

In practice. Simulateur : appel à 1 s, LED couloir à 1 Hz et bip 200 ms à 1,0/6,0/11,0 s ; URGENCE à 61 s (buzzer continu, LED 4 Hz) ; présence à 65 s (LED verte, silence) ; présence à 66 s (REPOS).

2 Midterm 2023–2024: transport et mobilité

2.1 Exercice 1 : QCM (2 pts)

Réponse

Q1 **b** Real-Time (freiner *à temps* est la contrainte) ; Q2 **b** ESP32 (bas coût, Wi-Fi et Bluetooth intégrés) ; Q3 **a** constante évaluée à la compilation ; Q4 **b** inclure/retirer le code de débogage à la compilation ; Q5 **b** nombre d'octets en attente ; Q6 **b** saute les non-numériques et lit le nombre ; Q7 **b** 3, 5, 6, 9, 10, 11 ; Q8 **c** un `main()` caché du cur Arduino.

2.2 Exercice 2 : tableau des types (1,5 pt)

Réponse

a) Sur l'ATmega328P (compilateur avr-gcc) :

Type	Octets	Plage
`bool`	1	`false` / `true` (0 / 1)
`byte`	1	0 à 255
`unsigned int`	2	0 à 65 535
`long`	4	−2 147 483 648 à 2 147 483 647
`unsigned long`	4	0 à 4 294 967 295
`float`	4	$\pm 3{,}4028235 \times 10^{38}$, ≈ 6–7 chiffres significatifs
`double`	4	identique à `float` sur l'UNO (pas de double précision)

(Pour mémoire : `int` = 2 octets, −32 768 à 32 767 ; `char` = 1 octet.)

b) `int` est signé sur 16 bits : dès que `millis()` dépasse 32 767 ms, soit ≈ 32,8 s, la valeur stockée

devient négative (puis fausse). Il faut `unsigned long tDepart = millis();` (le type de retour de `millis()`, 4 octets, débordement après ≈ 49,7 jours, et les soustractions `millis() - tDepart` restent justes même au débordement).

2.3 Exercice 3 : ordonner et compléter (1,5 pt)

Réponse

a) C → B → E → D → A → F : créer et renommer le circuit ; placer l'UNO, les trois LEDs, les trois résistances de 220 Ω et la breadboard ; câbler chaque LED en série avec sa résistance entre une broche et GND ; ouvrir l'éditeur texte et écrire le sketch ; lancer la simulation et observer ; corriger si la séquence est fausse.
b)

```
int main() {
  init();          // configuration interne de la carte (timers, millis, ADC...)
  setup();         // (1) une seule fois
  for (;;) {       // (2) boucle infinie
    loop();        // (3) a chaque tour (le coeur appelle aussi serialEventRun() si defini)
  }
}
```

2.4 Exercice 4 : calculs, éclairage d'une trottinette (2 pts)

Réponse

a) $R = (5 - 3{,}2)/0{,}015 = 120\,\Omega$: valeur E12 exacte, 120 Ω (choisir 150 Ω si l'on veut une marge : $I = 12\,\text{mA}$).
b) $R = (5 - 1{,}8)/0{,}010 = 320\,\Omega$ → valeur normalisée supérieure 330 Ω ; courant réel $I = 3{,}2/330 = 9{,}7\,\text{mA}$ (avec 270 Ω : 11,9 mA, au-dessus de la cible).
c) Non : quatre LEDs de 15 mA en parallèle sur une broche demandent $4 \times 15 = 60\,\text{mA}$, au-delà de la limite absolue (40 mA) et du triple de la valeur recommandée (20 mA) : broche dégradée ou détruite. Solutions : un transistor (NPN ou MOSFET) commandé par la broche et alimentant les LEDs depuis 5 V ; ou une broche par LED (15 mA chacune) ; ou des LEDs en série avec une alimentation plus haute via un driver.
d) Bouton appuyé : la résistance de pull-up est entre 5 V et GND : $I = 5/10\,000 = 0{,}5\,\text{mA}$ (négligeable). Avec 100 Ω : $I = 5/100 = 50\,\text{mA}$, $P = 5 \times 0{,}05 = 0{,}25\,\text{W}$: la résistance chauffe à sa limite, l'alimentation 5 V est sollicitée inutilement, et tant que le bouton est appuyé on gaspille plus que tout le reste du montage. Une pull-up doit être faible (10 kΩ typique).

2.5 Exercice 5 : montage à compléter, clignotants de vélo (2,5 pts)

Lecture du schéma : les fils 1 et 2 (bleus) arrivent sur les colonnes des deux boutons (colonnes 5 et 11), dont l'autre patte est reliée au rail − par un cavalier ; les fils 3 et 4 (orange) arrivent aux résistances (col. 14 et 20) en série avec LED G et LED D dont les cathodes vont au rail − ; le fil 5 (violet) arrive sur la borne + du buzzer (col. 26), sa borne − au rail ; le fil 6 (noir) relie le rail −.

Réponse

a)

Fil	Relié à	Broche
1	bouton gauche (patte libre ; l'autre au rail GND)	D2 (`BP_GAUCHE`)
2	bouton droite	D3 (`BP_DROITE`)
3	résistance → anode LED G (cathode au GND)	D5 (`LED_GAUCHE`)
4	résistance → anode LED D	D6 (`LED_DROITE`)
5	borne + du buzzer (borne − au GND)	D8 (`BUZZER`)
6	rail − de la breadboard	GND

b) Aucune résistance externe et bouton vers GND : `INPUT_PULLUP` (pull-up interne). Appuyé : `LOW`.
c) (1) `INPUT_PULLUP` ; (2) `INPUT_PULLUP` ; (3) `OUTPUT` ; (4) `OUTPUT` ; buzzer `OUTPUT` ; (5) `LOW`.

2.6 Exercice 6 : débogage, passage à niveau (2 pts)

Réponse

Six défauts (cinq suffisent) :

1. (A) `int tDebut` : `millis()` renvoie un `unsigned long` ; en `int` la valeur déborde après 32,8 s et la soustraction est fausse. → `unsigned long tDebut = 0;`
2. (B) `pinMode(CAPTEUR, INPUT)` avec un contact vers GND : au repos la broche est *flottante* (déclenchements aléatoires). → `INPUT_PULLUP` (repos HIGH, train = LOW).
3. (C) `pinMode(SONNERIE, OUTPUT)` manque : `digitalWrite(SONNERIE, HIGH)` n'active que le pull-up d'une entrée, la sonnerie ne sonne pas. → l'ajouter dans `setup()`.
4. (D) `if (digitalRead(CAPTEUR) = LOW)` : affectation, l'expression vaut 0, jamais vraie (et écrase rien d'utile). → `==`.
5. (E) `if (alarme);` : le point-virgule termine le `if`, les lignes suivantes s'exécutent toujours. → supprimer le `;` et mettre un bloc `{ }`.
6. (F) `delay(8000)` bloque 8 s : plus de lecture du capteur, pas de clignotement du feu, consigne non bloquant violée. → comparer `millis() - tDebut >= 8000` et éteindre feu et sonnerie à l'échéance (ils ne sont jamais éteints dans l'original).

```
// Midterm 2023-2024, Exercice 6 : passage a niveau, version corrigee (5 erreurs)
const int CAPTEUR = 2;
const int FEU = 8, SONNERIE = 9;
const unsigned long T_ALARME = 8000, T_CLIGNO = 500;
unsigned long tDebut = 0, tCligno = 0;      // (A) unsigned long, pas int
bool alarme = false, feu = false;

void setup() {
  pinMode(CAPTEUR, INPUT_PULLUP);           // (B) contact vers GND : pull-up interne, sinon entree flottante
  pinMode(FEU, OUTPUT);
  pinMode(SONNERIE, OUTPUT);                // (C) sortie manquante
}

void loop() {
  if (!alarme && digitalRead(CAPTEUR) == LOW) {   // (D) == et non =
    alarme = true; tDebut = millis(); tCligno = millis(); feu = true;
  }
  if (alarme) {                             // (E) pas de ';' apres le if
    if (millis() - tCligno >= T_CLIGNO) { tCligno = millis(); feu = !feu; }
    digitalWrite(FEU, feu);                 // feu rouge clignotant
    digitalWrite(SONNERIE, HIGH);
    if (millis() - tDebut >= T_ALARME) {    // (F) millis() au lieu de delay(8000)
      alarme = false;
      digitalWrite(FEU, LOW); digitalWrite(SONNERIE, LOW);
```

```
    }
  }
}
```

2.7 Exercice 7 : trace d'exécution, afficheur de quai (1,5 pt)

Réponse

a) `Serial.parseInt()` saute les caractères non numériques en attendant un chiffre (1 s au plus), lit le nombre, puis *s'arrête sur le caractère suivant sans le consommer*. Trace :

- `Q3\n` : `type='Q'`, `parseInt` lit 3 et laisse `'\n'` → **Quai 3**. Tour suivant : `type='\n'`, `parseInt` attend 1 s, renvoie 0 → **Commande inconnue**.
- `R12\n` : **Retard : 12 min**, puis, faute de `break`, on tombe dans `case 'A'` : **Annule**. Puis le `'\n'` restant → **Commande inconnue**.
- `A\n` : `type='A'`, `parseInt` *consomme* le `'\n'` (non numérique) en attendant un chiffre, expire au bout d'1 s et renvoie 0 → **Annule** (après une seconde de latence). Rien ne reste dans le tampon.

Six lignes au total : `Quai 3 / Commande inconnue / Retard : 12 min / Annule / Commande inconnue / Annule`.

b) Défaut 1 : `break` manquant après `case 'R'` (fall-through vers `'A'`). Défaut 2 : le `'\n'` de fin de ligne est traité comme une commande (et `parseInt` y perd 1 s) ; il faut ignorer les caractères blancs avant de lire le type (ou utiliser `Serial.readStringUntil('\n')`).

```
// Midterm 2023-2024, Exercice 7 : afficheur de quai, version corrigee
void setup() { Serial.begin(9600); }
void loop() {
  if (Serial.available() > 0) {
    char type = Serial.read();
    if (type == '\n' || type == '\r' || type == ' ') return;   // defaut 2 : ignorer le '\n' restant
    int valeur = Serial.parseInt();
    switch (type) {
      case 'Q': Serial.print("Quai "); Serial.println(valeur); break;
      case 'R': Serial.print("Retard : "); Serial.print(valeur); Serial.println(" min"); break;  // defaut 1 :
     break manquant
      case 'A': Serial.println("Annule"); break;
      default:  Serial.println("Commande inconnue");
    }
  }
}
```

In practice. Simulateur : l'original produit exactement les six lignes ci-dessus ; le corrigé produit `Quai 3 / Retard : 12 min / Annule`.

2.8 Exercice 8 : analyse de la FSM du passage à niveau (3 pts)

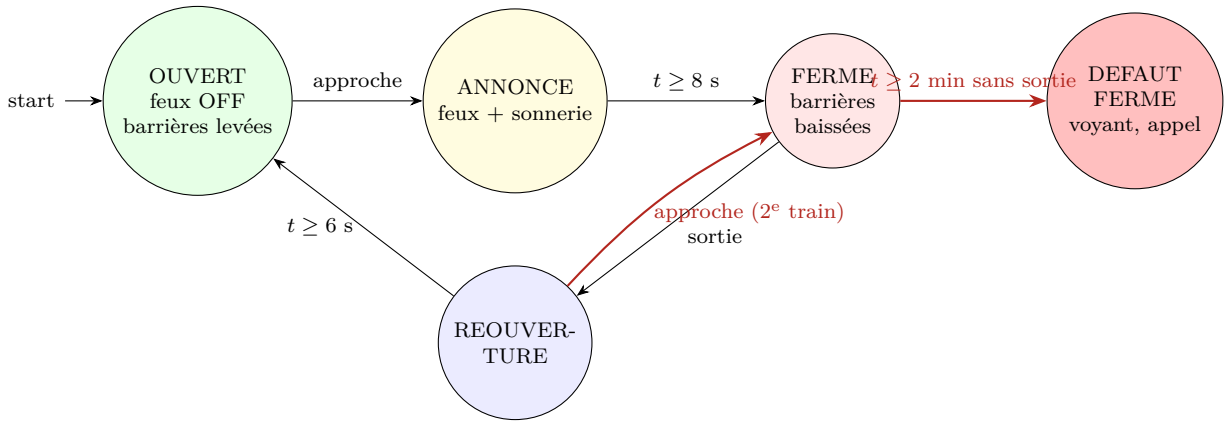
Réponse

a)

État courant	Événement	État suivant	Action
OUVERT	approche	ANNONCE	feux ON, sonnerie ON
ANNONCE	$t \geq 8\,\text{s}$	FERME	barrières baissées (sonnerie OFF)
FERME	sortie	REOUVERTURE	barrières montent
REOUVERTURE	$t \geq 6\,\text{s}$	OUVERT	feux OFF, barrières levées

b) Train détecté à $t = 0$: ANNONCE de 0 à 8 s, FERME de 8 à 45 s, REOUVERTURE de 45 à

51 s, OUVERT ensuite. Donc $t = 5$ s : **ANNONCE** ; $t = 20$ s : **FERME** ; $t = 48$ s : **REOUVERTURE** ; $t = 55$ s : **OUVERT**.
c) (1) Pendant REOUVERTURE aucun événement *approche* n'est prévu : un second train arrive, la FSM finit de lever les barrières et passe en OUVERT avec un train sur la voie, puis seulement repasse par ANNONCE (8 s de feux) : barrières ouvertes au passage du train. Correction : REOUVERTURE $\xrightarrow{\text{approche}}$ FERME (on referme immédiatement, sans attendre la fin de la montée). (2) Capteur de sortie en panne : la FSM reste en FERME indéfiniment. C'est l'état *sûr* (barrières fermées), donc pas de danger ferroviaire, mais la route est bloquée sans que personne ne soit prévenu ; le risque réel est que des conducteurs contournent les barrières. Correction : temporisation de surveillance dans FERME ($t \geq 2$ min sans *sortie*) vers un état DEFAUT_FERME qui garde les barrières baissées, allume un voyant et appelle la maintenance ; jamais d'ouverture automatique sans confirmation (capteur redondant ou acquittement humain).



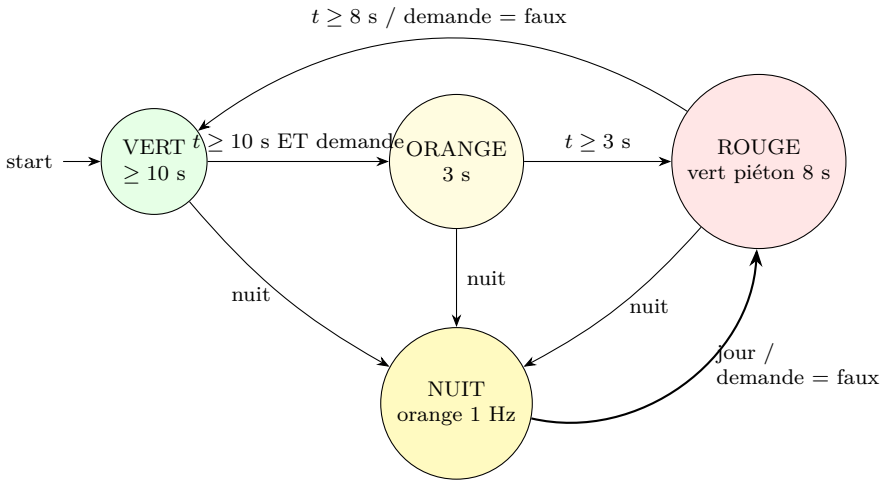

```ino
// Midterm 2023-2024, Exercice 8 : passage a niveau, FSM corrigee (second train, panne du capteur de sortie)
const int APPROCHE = 2, SORTIE = 3;                  // contacts vers GND, INPUT_PULLUP, LOW = actif
const int FEUX = 8, SONNERIE = 9, BARRIERE = 10, DEFAUT = 11;   // BARRIERE HIGH = baissee
const unsigned long T_ANNONCE = 8000, T_REOUV = 6000, T_MAX_FERME = 120000UL;
enum class Etat { OUVERT, ANNONCE, FERME, REOUVERTURE, DEFAUT_FERME };
Etat etat = Etat::OUVERT;
unsigned long tEntree = 0;
bool approche() { return digitalRead(APPROCHE) == LOW; }
bool sortie()   { return digitalRead(SORTIE) == LOW; }
void changer(Etat e) { etat = e; tEntree = millis(); }
void setup() {
  pinMode(APPROCHE, INPUT_PULLUP); pinMode(SORTIE, INPUT_PULLUP);
  pinMode(FEUX, OUTPUT); pinMode(SONNERIE, OUTPUT); pinMode(BARRIERE, OUTPUT); pinMode(DEFAUT, OUTPUT);
}
void loop() {
  unsigned long ecoule = millis() - tEntree;
  switch (etat) {
    case Etat::OUVERT:
      digitalWrite(FEUX, LOW); digitalWrite(SONNERIE, LOW); digitalWrite(BARRIERE, LOW);
      if (approche()) changer(Etat::ANNONCE);
      break;
    case Etat::ANNONCE:
      digitalWrite(FEUX, HIGH); digitalWrite(SONNERIE, HIGH);
      if (ecoule >= T_ANNONCE) changer(Etat::FERME);
      break;
    case Etat::FERME:
      digitalWrite(BARRIERE, HIGH); digitalWrite(SONNERIE, LOW);
      if (sortie()) changer(Etat::REOUVERTURE);
      else if (ecoule >= T_MAX_FERME) changer(Etat::DEFAUT_FERME);   // capteur de sortie en panne : on reste ferme, on signale
      break;
    case Etat::REOUVERTURE:
```

```
      if (approche()) changer(Etat::FERME);        // second train : on referme AVANT d'avoir fini d'ouvrir (etat sur)
      else if (ecoule >= T_REOUV) { digitalWrite(BARRIERE, LOW); changer(Etat::OUVERT); }
      break;
    case Etat::DEFAUT_FERME:                       // barrieres fermees, feux allumes, voyant defaut : intervention humaine
      digitalWrite(DEFAUT, HIGH); digitalWrite(FEUX, HIGH);
      break;
  }
}
```

2.9 Exercice 9 : feu tricolore avec bouton piéton et mode nuit (4 pts)

Réponse

b) L'appui est un événement bref (quelques dizaines de ms) qui peut survenir pendant les 10 s minimum de VERT, pendant ORANGE ou ROUGE ; s'il n'est pas mémorisé dans une variable (`demandePieton`), il est perdu avant que la condition $t \geq 10$ s et demande soit évaluée. La variable transforme un événement fugitif en *demande en attente*, effacée seulement quand elle a été servie (fin de ROUGE).
d) En sortant du mode nuit toutes les LEDs étaient hors séquence (orange clignotant) ; repartir de ROUGE arrête d'abord les voitures et donne aux piétons un passage, puis le cycle normal reprend : c'est l'état sûr. Repartir de VERT ferait passer de prudence à voie libre sans rouge intermédiaire, et une demande piéton en attente serait servie plus tard.



bouton piéton (tout état sauf NUIT) : demande = vrai

```
// Midterm 2023-2024, Exercice 9 : feu tricolore avec bouton pieton et mode nuit (sketch complet)
const int LED_V = 8, LED_O = 9, LED_R = 10, LED_PIETON = 11;
const int BTN_PIETON = 2, SW_NUIT = 4;             // vers GND, INPUT_PULLUP, LOW = actif
const unsigned long T_VERT_MIN = 10000, T_ORANGE = 3000, T_ROUGE = 8000, T_CLIGNO = 500, T_DEBOUNCE = 50;

enum class Etat { VERT, ORANGE, ROUGE, NUIT };
Etat etat = Etat::VERT;
unsigned long tEntree = 0, tCligno = 0, tChange = 0;
bool demandePieton = false, orange = false;        // b) l'appui est memorise ici
int lecturePrec = HIGH, etatStable = HIGH;

bool nuit() { return digitalRead(SW_NUIT) == LOW; }
bool appuiPieton() {                               // vrai une fois par appui (anti-rebond)
  int l = digitalRead(BTN_PIETON);
```

```
  if (l != lecturePrec) { tChange = millis(); lecturePrec = l; }
  if (millis() - tChange > T_DEBOUNCE && l != etatStable) { etatStable = l; return l == LOW; }
  return false;
}
void feux(bool v, bool o, bool r, bool p) {
  digitalWrite(LED_V, v); digitalWrite(LED_O, o); digitalWrite(LED_R, r); digitalWrite(LED_PIETON, p);
}
void changer(Etat e) { etat = e; tEntree = millis(); }

void setup() {
  pinMode(LED_V, OUTPUT); pinMode(LED_O, OUTPUT); pinMode(LED_R, OUTPUT); pinMode(LED_PIETON, OUTPUT);
  pinMode(BTN_PIETON, INPUT_PULLUP); pinMode(SW_NUIT, INPUT_PULLUP);
  changer(Etat::VERT);
}

void loop() {
  if (appuiPieton()) demandePieton = true;          // memorise meme pendant les 10 s
  unsigned long now = millis(), ecoule = now - tEntree;
  if (etat != Etat::NUIT && nuit()) changer(Etat::NUIT);   // depuis n'importe quel etat
  switch (etat) {
    case Etat::VERT:
      feux(1, 0, 0, 0);
      if (ecoule >= T_VERT_MIN && demandePieton) changer(Etat::ORANGE);
      break;
    case Etat::ORANGE:
      feux(0, 1, 0, 0);
      if (ecoule >= T_ORANGE) changer(Etat::ROUGE);
      break;
    case Etat::ROUGE:
      feux(0, 0, 1, 1);
      if (ecoule >= T_ROUGE) { demandePieton = false; changer(Etat::VERT); }   // demande effacee
      break;
    case Etat::NUIT:
      if (now - tCligno >= T_CLIGNO) { tCligno = now; orange = !orange; }
      feux(0, orange, 0, 0);
      if (!nuit()) { demandePieton = false; changer(Etat::ROUGE); }            // d) on repart de ROUGE
      break;
  }
}
```

In practice. Simulateur : appui piéton à 3 s ; ORANGE à 10 s, ROUGE + vert piéton à 13 s, VERT à 21 s ; interrupteur nuit à 25 s (orange 1 Hz), retour jour à 30 s → ROUGE.

3 Midterm 2024–2025: industrie, agriculture et énergie

3.1 Exercice 1 : questions à réponse courte (2 pts)

Réponse

1. Coût unitaire, consommation d'énergie, contraintes temps réel, ressources limitées (mémoire, CPU), taille/poids, fiabilité et sûreté, environnement (température, vibrations).
2. Stand-Alone, Real-Time, Networked, Mobile.
3. Rendre un microcontrôleur immédiatement utilisable : régulation d'alimentation, interface USB de programmation et de communication, oscillateur, bootloader, connecteurs vers les broches, LED et bouton reset ; on prototype sans concevoir de carte.
4. 8 bits, architecture AVR (RISC, Harvard : Flash programme séparée de la SRAM données), horloge 16 MHz sur l'UNO.
5. Rapport $3\,\text{mA}/0{,}1\,\mu\text{A} = 30\,000$: un capteur qui passe l'essentiel du temps en veille consomme des milliers de fois moins ; l'autonomie sur batterie est dominée par la part de temps actif (rapport cyclique), d'où le réveil périodique de l'exercice 6.

6. `#define` : substitution textuelle par le préprocesseur, sans type, sans portée, invisible au débogueur, erreurs de compilation obscures ; `const int` : constante typée, vérifiée par le compilateur, avec portée, visible au débogueur. (Même code machine en pratique.)
7. -1.
8. 40 mA maximum absolu par broche (et 200 mA pour l'ensemble des broches) ; 20 mA recommandés.

3.2 Exercice 2 : schéma de principe de la serre (1,5 pt)

Réponse

Capteurs (entrées) : capteur d'humidité du sol, flotteur de la cuve. Unité de traitement : ATmega328P. Actionneurs (sorties) : relais de la pompe, LED d'alarme. Mémoire non volatile : EEPROM (seuils). Communication : module Wi-Fi. Alimentation : batterie et panneau solaire.

3.3 Exercice 3 : QCM (2 pts)

Réponse

Q1 **b** Networked ; Q2 **c** STM32 (ARM 32 bits, nombreux périphériques, temps réel industriel) ; Q3 **b** EEPROM ; Q4 **b** 20 octets (10×2) ; Q5 **b** la valeur de `x` (déréférencement) ; Q6 **b** nommer des constantes entières (états) ; Q7 **b** états, événements, transitions, actions ; Q8 **a** table de transitions avec pointeurs de fonctions.

3.4 Exercice 4 : montage à compléter, station de pompage (2,5 pts)

Lecture du schéma : fil 1 (bleu) → colonne 5 du flotteur HAUT, dont l'autre patte (col. 2) va au rail + par un cavalier rouge, et une résistance de 10 kΩ relie la colonne 5 au rail − : *pull-down externe*. Fil 2 (bleu) → colonne 14 du flotteur BAS, dont l'autre patte va au rail −, sans résistance : *pull-up interne*. Fil 3 (vert) → broche IN du module relais (VCC au rail +, GND au rail −). Fil 4 (orange) → résistance → anode de la LED alarme (cathode au rail −). Fils 5 et 6 : rails + et − du bas (reliés aux rails du haut par les fils rouge et noir de droite).

Réponse

a) Fil 1 → D2 (`FLOT_HAUT`) ; 2 → D4 (`FLOT_BAS`) ; 3 → D7 (`RELAIS`) ; 4 → D9 (`LED_ALARME`) ; 5 → 5V ; 6 → GND.
b) Flotteur HAUT : montage *pull-down externe* (10 kΩ vers GND, contact vers 5 V) : `pinMode(FLOT_HAUT, INPUT);` contact fermé (cuve pleine) = `HIGH`, ouvert = `LOW`. Flotteur BAS : *pull-up interne* (contact vers GND) : `pinMode(FLOT_BAS, INPUT_PULLUP);` contact fermé (eau au-dessus du bas) = `LOW`, ouvert (cuve vide) = `HIGH`.
c)

```
bool cuvePleine = (digitalRead(FLOT_HAUT) == HIGH);   // contact ferme, pull-down
bool cuveVide   = (digitalRead(FLOT_BAS)  == HIGH);   // contact ouvert, pull-up
if (cuveVide)        digitalWrite(RELAIS, HIGH);      // la pompe demarre quand le bas est
    decouvert
else if (cuvePleine) digitalWrite(RELAIS, LOW);       // et s'arrete quand le haut est atteint
// entre les deux niveaux on ne change rien : hysteresis, pas de battement de la pompe
```

3.5 Exercice 5 : structures, pointeurs et références (2 pts)

Réponse

a) `N = sizeof(zones)/sizeof(zones[0]) = 3`. `plusSeche` renvoie l'adresse de la zone d'humidité minimale : Tomates (30). `arroser(*z, 20)` reçoit `zones[0]` *par référence* : son humidité devient 50. Le second `plusSeche` compare 50, 55, 42 : Fraises. Affichage :

```
3
Tomates
50
Fraises
```

b) Avec `void arroser(Zone z, int dose)` la fonction reçoit une *copie* de `zones[0]` ; `z.humidite += dose` modifie la copie, détruite au retour ; `zones[0].humidite` vaut toujours 30 : la troisième ligne afficherait **30** (et la quatrième Tomates). La référence (`&`) ou un pointeur sont nécessaires pour modifier la variable de l'appelant.
c) (1) `0` ; (2) `n` ; (3) `tab[i].humidite`.

3.6 Exercice 6 : bilan énergétique du nud capteur (1,5 pt)

Réponse

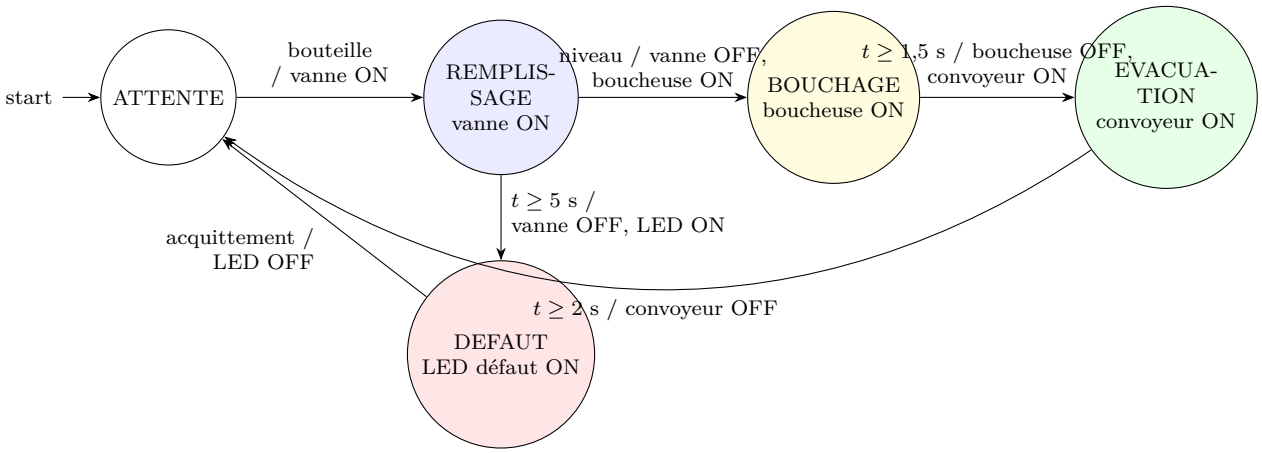
a) Cycle de 600 s : actif 20 mA × 2 s = 40 mAůs ; veille 0,05 mA × 598 s = 29,9 mAůs ; total 69,9 mAůs. Courant moyen $I_{\text{moy}} = 69{,}9/600 = 0{,}1165$ mA $\approx 0{,}117$ mA (117 ţA).
b) Autonomie = 2000 mAh/0,1165 mA ≈ 17 170 h ≈ 715 jours ≈ 2 ans (sans autodécharge ni pertes du régulateur).
c) Sans veille : 2000/20 = 100 h ≈ 4,2 jours. Conclusion : la gestion de la veille est le facteur dominant (rapport ≈ 170) ; sur batterie, le logiciel de mise en veille compte plus que le choix de la batterie.

3.7 Exercice 7 : retour sur les labs (1,5 pt)

Réponse

1. Avec `delay()` le processeur est bloqué pendant chaque phase (plusieurs secondes) : le bouton n'est lu qu'entre deux `delay`, un appui bref est perdu et la réaction attend la fin de la phase. La FSM avec `millis()` relit le bouton à chaque tour de `loop()` (des milliers de fois par seconde) et peut mémoriser la demande et réagir au bon moment.
2. Chaque LED a sa propre résistance entre sa broche et GND : les trois branches sont indépendantes, le courant de chacune ($I = (5 - V_F)/R$) ne dépend pas des autres. Avec une seule résistance commune : les LEDs se partagent le courant, la luminosité baisse dès qu'une deuxième s'allume, et comme les V_F diffèrent (rouge 1,8 V, verte/bleue >2 V) la LED à plus faible seuil accapare le courant : luminosités inégales, voire une LED qui ne s'allume pas.
3. Ce sont des *traces de débogage* : elles montrent dans quel état la FSM se trouve et quand elle change, pour vérifier le comportement sans oscilloscope. Pour les retirer sans les effacer : les encadrer par `#ifdef DEBUG ... #endif` (ou une macro `DBG(x)` vide quand `DEBUG` n'est pas défini) : le préprocesseur les exclut de la version finale, et elles reviennent en définissant `DEBUG`.

3.8 Exercice 8 : de la table de transitions au code, remplissage de bouteilles (3 pts)



Réponse

b) REMPLISSAGE → DEFAUT est un *chien de garde* : si le niveau n'est pas atteint en 5 s, quelque chose ne va pas (bouteille absente ou percée, vanne bloquée, capteur de niveau en panne). Sans elle, un capteur de niveau défaillant laisserait la vanne ouverte indéfiniment : la bouteille déborde, l'huile se répand sur la ligne, et la FSM reste bloquée en REMPLISSAGE sans jamais signaler le problème.

c) Énumération et `loop()` ci-dessous : les actions de la table sont exécutées *sur la transition*, l'instant d'entrée `tEntree` sert aux trois temporisations, les entrées sont actives à `LOW`.

```
// Midterm 2024-2025, Exercice 8 : remplissage de bouteilles (enum + loop complet)
const int BOUTEILLE = 2, NIVEAU = 3, ACQUIT = 4;   // INPUT_PULLUP, LOW = actif
const int VANNE = 8, BOUCHEUSE = 9, CONVOYEUR = 10, LED_DEFAUT = 11;
const unsigned long T_MAX_REMPL = 5000, T_BOUCHAGE = 1500, T_EVAC = 2000;

enum class Etat { ATTENTE, REMPLISSAGE, BOUCHAGE, EVACUATION, DEFAUT };
Etat etat = Etat::ATTENTE;
unsigned long tEntree = 0;
bool actif(int pin) { return digitalRead(pin) == LOW; }
void changer(Etat e) { etat = e; tEntree = millis(); }

void setup() {
  pinMode(BOUTEILLE, INPUT_PULLUP); pinMode(NIVEAU, INPUT_PULLUP); pinMode(ACQUIT, INPUT_PULLUP);
  pinMode(VANNE, OUTPUT); pinMode(BOUCHEUSE, OUTPUT); pinMode(CONVOYEUR, OUTPUT); pinMode(LED_DEFAUT, OUTPUT);
}

void loop() {
  unsigned long ecoule = millis() - tEntree;
  switch (etat) {
    case Etat::ATTENTE:
      if (actif(BOUTEILLE)) { digitalWrite(VANNE, HIGH); changer(Etat::REMPLISSAGE); }
      break;
    case Etat::REMPLISSAGE:
      if (actif(NIVEAU)) { digitalWrite(VANNE, LOW); digitalWrite(BOUCHEUSE, HIGH); changer(Etat::BOUCHAGE); }
      else if (ecoule >= T_MAX_REMPL) { digitalWrite(VANNE, LOW); digitalWrite(LED_DEFAUT, HIGH); changer(Etat::
     DEFAUT); }
      break;
    case Etat::BOUCHAGE:
      if (ecoule >= T_BOUCHAGE) { digitalWrite(BOUCHEUSE, LOW); digitalWrite(CONVOYEUR, HIGH); changer(Etat::
     EVACUATION); }
      break;
    case Etat::EVACUATION:
      if (ecoule >= T_EVAC) { digitalWrite(CONVOYEUR, LOW); changer(Etat::ATTENTE); }
      break;
    case Etat::DEFAUT:
      if (actif(ACQUIT)) { digitalWrite(LED_DEFAUT, LOW); changer(Etat::ATTENTE); }
```

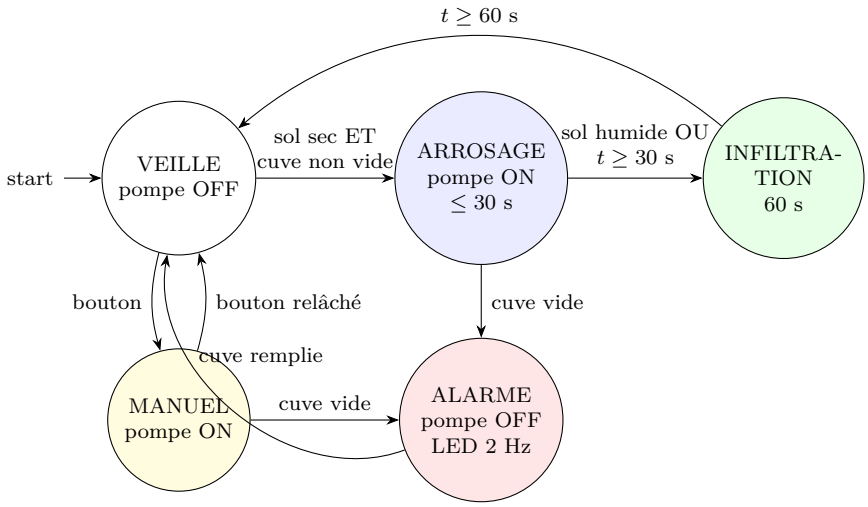
```
      break;
  }
}
```

In practice. Simulateur : bouteille à 1 s (vanne ON), niveau à 3 s (boucheuse 1,5 s, puis convoyeur 2 s, puis ATTENTE) ; deuxième bouteille à 8 s sans niveau : DEFAUT à 13 s, LED allumée ; acquittement à 14 s.

3.9 Exercice 9 : conception de la FSM d'arrosage (4 pts)

Réponse

a) `solSec = (digitalRead(D2) == HIGH)` ; `cuveVide = (digitalRead(D3) == HIGH)` (pull-up, flotteur ouvert = vide) ; `boutonManuel = (digitalRead(D4) == LOW)`.
d) Agronomique : l'eau met du temps à s'infiltrer jusqu'aux racines ; mesurer tout de suite après l'arrosage donnerait un sol de surface humide puis sec à nouveau (ou l'inverse), et arroser sans pause lessive le sol et noie les racines. Technique : la pause évite les *oscillations* (ARROSAGE ↔ VEILLE à chaque tour quand le capteur hésite autour du seuil) et laisse la pompe refroidir ; c'est une hystérésis temporelle.




```
// Midterm 2024-2025, Exercice 9 : arrosage automatique de la serre (sketch complet, non bloquant)
const int CAPT_SOL = 2, FLOTTEUR = 3, BTN_MANUEL = 4, POMPE = 7, LED_ALARME = 9;
const unsigned long T_ARROSAGE_MAX = 30000, T_INFILTRATION = 60000UL, T_CLIGNO = 250;   // 2 Hz = 250 ms

enum class Etat { VEILLE, ARROSAGE, INFILTRATION, MANUEL, ALARME };
Etat etat = Etat::VEILLE;
unsigned long tEntree = 0, tCligno = 0;
bool led = false;

bool solSec()       { return digitalRead(CAPT_SOL) == HIGH; }     // a)
bool cuveVide()     { return digitalRead(FLOTTEUR) == HIGH; }
bool boutonManuel() { return digitalRead(BTN_MANUEL) == LOW; }
void changer(Etat e) { etat = e; tEntree = millis(); }

void setup() {
  pinMode(CAPT_SOL, INPUT); pinMode(FLOTTEUR, INPUT_PULLUP); pinMode(BTN_MANUEL, INPUT_PULLUP);
  pinMode(POMPE, OUTPUT); pinMode(LED_ALARME, OUTPUT);
}

void loop() {
  unsigned long now = millis(), ecoule = now - tEntree;
  switch (etat) {
```

```
    case Etat::VEILLE:
      digitalWrite(POMPE, LOW); digitalWrite(LED_ALARME, LOW);
      if (boutonManuel()) changer(Etat::MANUEL);
      else if (solSec() && !cuveVide()) changer(Etat::ARROSAGE);
      break;
    case Etat::ARROSAGE:
      digitalWrite(POMPE, HIGH);
      if (cuveVide()) changer(Etat::ALARME);                    // marche a sec interdite
      else if (!solSec() || ecoule >= T_ARROSAGE_MAX) changer(Etat::INFILTRATION);
      break;
    case Etat::INFILTRATION:
      digitalWrite(POMPE, LOW);
      if (ecoule >= T_INFILTRATION) changer(Etat::VEILLE);
      break;
    case Etat::MANUEL:
      digitalWrite(POMPE, HIGH);
      if (cuveVide()) changer(Etat::ALARME);
      else if (!boutonManuel()) changer(Etat::VEILLE);
      break;
    case Etat::ALARME:
      digitalWrite(POMPE, LOW);
      if (now - tCligno >= T_CLIGNO) { tCligno = now; led = !led; digitalWrite(LED_ALARME, led); }
      if (!cuveVide()) changer(Etat::VEILLE);
      break;
  }
}
```

In practice. Simulateur : sol sec et cuve pleine au départ → pompe ON ; sol humide à 5 s → INFILTRATION 60 s ; manuel à 65,2 s (pompe suit le bouton) ; sol sec à 68 s → ARROSAGE ; cuve vide à 70 s → ALARME (pompe OFF, LED 2 Hz) ; cuve remplie à 74 s → VEILLE puis ARROSAGE.

4 Midterm 2 (21 octobre 2026): industrie, automobile, surveillance

4.1 Exercice 1 : QCM (2 pts)

Réponse

Q1 **b** ; Q2 **b** Real-Time (l'airbag doit se déclencher en quelques ms) ; Q3 **c** Networked ; Q4 **b** 8 bits AVR RISC à 16 MHz ; Q5 **c** Flash 32 Ko ; Q6 **a** un binaire `.hex` ; Q7 **c** 64 octets ; Q8 **b** 40 mA max, 20 mA recommandés.

4.2 Exercice 2 : compléter les concepts (1,5 pt)

Réponse

(1) `setup()` ; (2) `loop()` ; (3) `main()` ; (4) flottante ; (5) `INPUT_PULLUP` ; (6) rebonds ; (7) états ; (8) transitions ; (9) `enum` ; (10) SRAM ; (11) EEPROM ; (12) référence. Non utilisés : `INPUT`, `struct`, Flash, valeur, `static`.

4.3 Exercice 3 : choix d'une plateforme (1,5 pt)

Réponse

a) ESP32 : Wi-Fi (et Bluetooth) intégré, donc envoi direct vers le tableau de bord sans module externe ; faible coût et modes veille basse consommation pour des nuds nombreux et simples (quelques capteurs de température).
b) STM32 : microcontrôleur ARM Cortex-M 32 bits rapide avec de nombreux périphériques (timers PWM, CAN, encodeurs, ADC) pour piloter moteurs et capteurs du bras ; déterminisme temps réel (interruptions, RTOS) et qualification industrielle, contrairement à un Raspberry Pi sous Linux.
c) Raspberry Pi : processeur et RAM d'un ordinateur sous Linux, nécessaires pour décoder et analyser plusieurs flux vidéo ; pile réseau complète et serveur web/HTTPS disponibles nativement (Python, OpenCV), hors de portée d'un microcontrôleur.

4.4 Exercice 4 : entrées/sorties d'une presse industrielle (2 pts)

Réponse

a) BP1 (vers GND, pull-up interne) : repos `HIGH`, appuyé `LOW`. FC (contact vers 5 V, 10 kΩ vers GND) : repos `LOW`, actionné `HIGH` ; c'est un montage *pull-down externe*.
b) $R = (5 - 2{,}1)/0{,}012 = 241{,}7\,\Omega \rightarrow 270\,\Omega$ (valeur E12 supérieure) ; courant réel $I = 2{,}9/270 = 10{,}7$ mA, inférieur aux 20 mA recommandés (220 Ω donnerait 13,2 mA, acceptable aussi mais au-dessus des 12 mA visés). D0 et D1 sont RX et TX de la liaison série : les utiliser perturbe le téléversement et le moniteur série, et une charge sur TX fausse les trames.
c) Le contact mécanique *rebondit* : à chaque activation le niveau oscille pendant 5 à 20 ms, et la boucle, qui tourne en microsecondes, compte chaque front comme une pièce (3 ou 4 au lieu d'une). Anti-rebond logiciel : on ne valide un nouveau niveau que s'il est resté stable 30 ms, et on ne renvoie `true` que sur le front montant stable.

```cpp
// Midterm 2 (2026), Exercice 4c : compteur de pieces, anti-rebond logiciel 30 ms sur le fin de course
const int BP1 = 2, FC = 3, LED_VERTE = 8, LED_ROUGE = 9;
const unsigned long T_DEBOUNCE = 30;
int fcStable = LOW, fcLecturePrec = LOW;   // pull-down externe : repos LOW, actionne HIGH
unsigned long fcChange = 0;
unsigned long pieces = 0;

bool fcActive() {                          // true une seule fois par activation
  int l = digitalRead(FC);
  if (l != fcLecturePrec) { fcChange = millis(); fcLecturePrec = l; }
  if (millis() - fcChange >= T_DEBOUNCE && l != fcStable) {
    fcStable = l;
    return l == HIGH;                      // front montant stable = une piece
  }
  return false;
}

void setup() {
  Serial.begin(9600);
  pinMode(BP1, INPUT_PULLUP); pinMode(FC, INPUT);
  pinMode(LED_VERTE, OUTPUT); pinMode(LED_ROUGE, OUTPUT);
}
void loop() {
  if (fcActive()) { pieces++; Serial.print("Pieces: "); Serial.println(pieces); }
}
```

In practice. Simulateur : activation à 1 s avec quatre rebonds de 3 ms, relâchement avec rebonds à 1,5 s, activation propre à 3 s : `Pieces: 1` puis `Pieces: 2`.

4.5 Exercice 5 : analyse de code (2,25 pts)

Réponse

Extrait A. Trois lignes : `1 1`, `1 2`, `1 3`. `a` est une variable locale automatique, recréée et remise à 0 à chaque appel ; `b` est `static` : initialisée une seule fois, elle conserve sa valeur entre les appels et compte les appels.
Extrait B. `sizeof(zones)` = 3 structures de deux `int` de 2 octets = **12** (sur l'UNO ; 24 sur un PC où `int` fait 4 octets). La boucle parcourt 12/4 = 3 zones et n'affiche que celles dont `active` est non nul : `2` puis `3`. (Le compilateur avertit : comparaison entre `int` et `size_t`, sans conséquence ici.)
Extrait C. `if (c = 'S')` est une *affectation* : `c` reçoit `'S'` et l'expression vaut `'S'` (non nul, donc vrai) : quelle que soit la lettre envoyée, y compris M , le moteur est mis à `LOW` (arrêt). Une fois corrigé en `==`, l'envoi de S avec Nouvelle ligne transmet *deux* caractères, `'S'` puis `'\n'` : le premier arrête le moteur, le second tombe dans le `else` et le redémarre aussitôt. Correction sûre : ignorer les caractères de fin de ligne (et les espaces) et n'accepter que des commandes explicites (`'S'` arrêt, `'M'` marche), jamais un `else` qui met en marche sur n'importe quoi.

```
// Midterm 2 (2026), Exercice 5, extrait C corrige : convoyeur commande par le moniteur serie
const int MOTEUR = 8;
void setup() { Serial.begin(9600); pinMode(MOTEUR, OUTPUT); }
void loop() {
  if (Serial.available() > 0) {
    char c = Serial.read();
    if (c == '\n' || c == '\r' || c == ' ') return;      // le '\n' de "Nouvelle ligne" n'est pas une commande
    if (c == 'S')      digitalWrite(MOTEUR, LOW);        // == et non =
    else if (c == 'M') digitalWrite(MOTEUR, HIGH);       // commande explicite de marche
  }
}
```

4.6 Exercice 6 : montage à compléter, trois boutons, trois LEDs (2,5 pts)

Lecture du schéma : BP1 (colonnes 2–5) : patte col. 2 au rail + (cavalier rouge), patte col. 5 au fil 1 et à une 10 kΩ vers le rail − : *pull-down externe*. BP2 (col. 11–14) : patte col. 11 au rail − (cavalier noir), patte col. 14 au fil 3 et à une 10 kΩ vers le rail + (cavalier rouge col. 17) : *pull-up externe*. BP3 (col. 20–23) : patte col. 20 au rail −, patte col. 23 au fil 5, aucune résistance : *pull-up interne*. Les fils 2, 4, 6 arrivent sur les résistances des LED1 (rouge), LED2 (jaune), LED3 (verte), cathodes au rail − ; 7 et 8 sont les rails.

Réponse

a)

Fil	Relié à	Broche
1	BP1 (patte côté 10 kΩ vers GND ; autre patte au 5 V)	D2 (`BP1`)
2	résistance → LED1 rouge	D3 (`LED1`)
3	BP2 (patte côté 10 kΩ vers 5 V ; autre patte au GND)	D4 (`BP2`)
4	résistance → LED2 jaune	D8 (`LED2`)
5	BP3 (autre patte au GND, sans résistance)	D9 (`BP3`)
6	résistance → LED3 verte	D10 (`LED3`)
7	rail +	5V
8	rail −	GND

b) BP1 : pull-down externe, relâché `LOW`, appuyé `HIGH`. BP2 : pull-up externe, relâché `HIGH`, appuyé

LOW. BP3 : pull-up interne, relâché HIGH, appuyé LOW.
c) (1) INPUT ; (2) INPUT ; (3) INPUT_PULLUP ; (4) HIGH ; (5) LOW ; (6) LOW.

```ino
// Midterm 2 (2026), Exercice 6 : trois boutons, trois LEDs (blancs completes)
const int BP1 = 2, BP2 = 4, BP3 = 9;
const int LED1 = 3, LED2 = 8, LED3 = 10;
void setup() {
  pinMode(BP1, INPUT);          // (1) pull-down externe 10 k vers GND, bouton vers 5 V
  pinMode(BP2, INPUT);          // (2) pull-up externe 10 k vers 5 V, bouton vers GND
  pinMode(BP3, INPUT_PULLUP);   // (3) pull-up interne, bouton vers GND
  pinMode(LED1, OUTPUT); pinMode(LED2, OUTPUT); pinMode(LED3, OUTPUT);
}
void loop() {
  digitalWrite(LED1, digitalRead(BP1) == HIGH ? HIGH : LOW);   // (4) appuye = HIGH
  digitalWrite(LED2, digitalRead(BP2) == LOW  ? HIGH : LOW);   // (5) appuye = LOW
  digitalWrite(LED3, digitalRead(BP3) == LOW  ? HIGH : LOW);   // (6) appuye = LOW
}
```

4.7 Exercice 7 : retour sur les labs (1 pt)

Réponse

1. LED_BUILTIN = D13. Deux clignotements par seconde = période 500 ms : remplacer les deux `delay(1000)` par `delay(250)` (ou, en non bloquant, basculer toutes les 250 ms avec `millis()`).
2. Les cinq trous d'une demi-rangée (a–e, f–j) sont reliés entre eux ; les rails horizontaux + et − sont reliés sur toute la longueur ; les deux moitiés sont séparées par la rainure centrale. Un bouton à cheval sur la rainure met ses deux paires de pattes sur des bandes différentes, sinon ses pattes seraient court-circuitées par la bande. Deux pattes d'une LED dans la même colonne seraient reliées entre elles : LED court-circuitée, jamais allumée.
3. Cycle OFF → ON → BLINK → OFF à chaque appui. `loop()` tourne des milliers de fois par seconde : tester l'*état* (bouton enfoncé) ferait avancer le cycle à chaque tour pendant toute la durée de l'appui ; on détecte le *front* (passage relâché → appuyé) pour compter un appui une seule fois. Sans anti-rebond, les rebonds mécaniques produisent plusieurs fronts par appui : la lampe saute des modes (OFF → BLINK, ou retour à OFF).

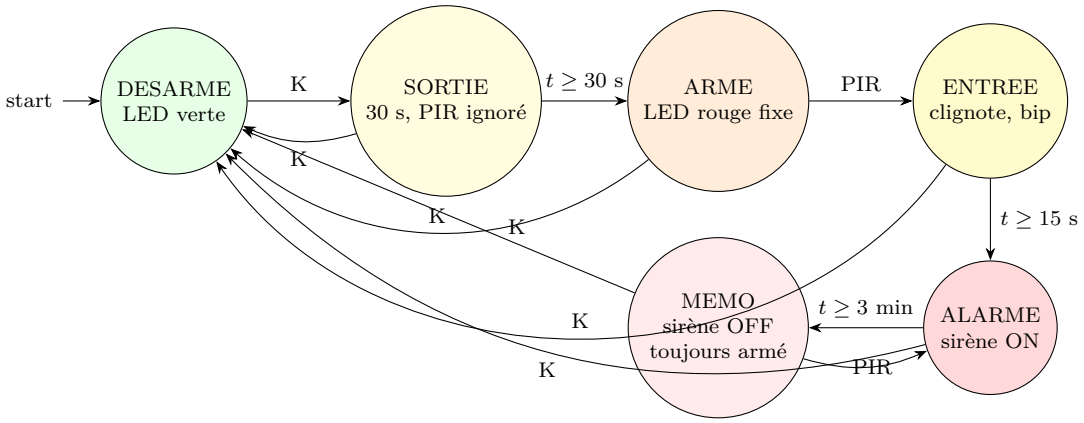
4.8 Exercice 8 : analyse de la FSM de l'alarme (2,5 pts)

Réponse

a) États : DESARME, ARME, ENTREE, ALARME. Événements : K (clé/code valide), PIR (détection de mouvement), $t \geq 15$ s. Sorties : LED verte ; LED rouge fixe ; LED rouge clignotante + bip ; sirène + LED clignotante.
b) K à 0 : ARME. PIR à 10 s : ENTREE. $t \geq 15$ s après 10 s : ALARME à 25 s. K à 40 s : DESARME. Donc $t = 5$: **ARME**, LED rouge fixe ; $t = 20$: **ENTREE**, LED rouge clignote + bip ; $t = 30$: **ALARME**, sirène ON, LED clignote ; $t = 45$: **DESARME**, LED verte.
c)

État courant	Événement	État suivant	Action
DESARME	K	ARME	LED verte OFF, LED rouge fixe
ARME	PIR	ENTREE	LED rouge clignote, bip
ARME	K	DESARME	LED verte
ENTREE	$t \geq 15\,\text{s}$	ALARME	sirène ON
ENTREE	K	DESARME	bip OFF, LED verte
ALARME	K	DESARME	sirène OFF, LED verte

d) Défaut : il n'y a pas de *délai de sortie*. Dès K, le PIR est actif ; l'agent qui quitte le local est détecté, la FSM passe en ENTREE et, 15 s plus tard, en ALARME, à moins qu'il ne revienne taper la clé. Il faut un état SORTIE (30 s, PIR ignoré, LED clignotante) entre K et ARME. Réglementation : ALARME doit couper la sirène au bout de 3 min : ajouter ALARME $\xrightarrow{t \geq 3\,\text{min}}$ MEMO (sirène OFF, intrusion mémorisée, système toujours armé : un nouveau PIR relance la sirène, K désarme). Diagramme corrigé :




```
// Midterm 2 (2026), Exercice 8d : alarme anti-intrusion corrigee (delai de sortie + arret sirene 3 min)
const int CLE = 2, PIR = 3, LED_V = 8, LED_R = 9, SIRENE = 10, BUZZER = 11;
const unsigned long T_SORTIE = 30000UL, T_ENTREE = 15000UL, T_SIRENE = 180000UL, T_CLIGNO = 500, T_DEBOUNCE =
    50;
enum class Etat { DESARME, SORTIE, ARME, ENTREE, ALARME, MEMO };
Etat etat = Etat::DESARME;
unsigned long tEntree = 0, tCligno = 0, tChange = 0;
int lecturePrec = HIGH, etatStable = HIGH; bool led = false;
bool cle() {                          // bouton a cle vers GND : un front = une action
  int l = digitalRead(CLE);
  if (l != lecturePrec) { tChange = millis(); lecturePrec = l; }
  if (millis() - tChange > T_DEBOUNCE && l != etatStable) { etatStable = l; return l == LOW; }
  return false;
}
bool pir() { return digitalRead(PIR) == HIGH; }
void changer(Etat e) { etat = e; tEntree = millis(); }
void setup() {
  pinMode(CLE, INPUT_PULLUP); pinMode(PIR, INPUT);
  pinMode(LED_V, OUTPUT); pinMode(LED_R, OUTPUT); pinMode(SIRENE, OUTPUT); pinMode(BUZZER, OUTPUT);
}
void loop() {
  bool k = cle(); unsigned long now = millis(), ecoule = now - tEntree;
  if (now - tCligno >= T_CLIGNO) { tCligno = now; led = !led; }
  switch (etat) {
    case Etat::DESARME:
      digitalWrite(LED_V, HIGH); digitalWrite(LED_R, LOW); digitalWrite(SIRENE, LOW); digitalWrite(BUZZER, LOW);
      if (k) changer(Etat::SORTIE);
      break;
    case Etat::SORTIE:                                  // delai de sortie : PIR ignore 30 s
      digitalWrite(LED_V, LOW); digitalWrite(LED_R, led); digitalWrite(BUZZER, LOW);
```

```
      if (k) changer(Etat::DESARME);
      else if (ecoule >= T_SORTIE) changer(Etat::ARME);
      break;
    case Etat::ARME:
      digitalWrite(LED_R, HIGH);
      if (k) changer(Etat::DESARME);
      else if (pir()) changer(Etat::ENTREE);
      break;
    case Etat::ENTREE:                                  // delai d'entree : 15 s pour taper la cle
      digitalWrite(LED_R, led); digitalWrite(BUZZER, led);
      if (k) changer(Etat::DESARME);
      else if (ecoule >= T_ENTREE) changer(Etat::ALARME);
      break;
    case Etat::ALARME:
      digitalWrite(SIRENE, HIGH); digitalWrite(LED_R, led); digitalWrite(BUZZER, LOW);
      if (k) changer(Etat::DESARME);
      else if (ecoule >= T_SIRENE) changer(Etat::MEMO); // reglementation : sirene coupee apres 3 min
      break;
    case Etat::MEMO:                                    // sirene coupee, intrusion memorisee, toujours arme
      digitalWrite(SIRENE, LOW); digitalWrite(LED_R, led);
      if (k) changer(Etat::DESARME);
      else if (pir()) changer(Etat::ALARME);
      break;
  }
}
```

4.9 Exercice 9 : alarme de magasin, montage et code (2,25 pts)

Lecture du schéma : le PIR a trois broches VCC, OUT, GND (colonnes 3, 4, 5) : VCC au rail + par un cavalier rouge, GND au rail −, OUT au fil 1. Les fils 2 et 3 arrivent sur les résistances de la LED verte et de la LED rouge, le fil 4 sur la borne + du buzzer ; 5 et 6 sont les rails.

Réponse

a) Fil 1 → D2 (`PIR`, sortie OUT) ; 2 → D11 (`LED_VERTE`) ; 3 → D12 (`LED_ROUGE`) ; 4 → D13 (`BUZZER`) ; 5 → 5V ; 6 → GND. Le HC-SR501 est un capteur *actif* : il contient un régulateur, un élément pyroélectrique et un amplificateur qui doivent être alimentés en permanence ; sa broche VCC va donc sur le rail 5 V, et seule sa sortie OUT (signal logique HIGH à la détection) va vers une broche de l'Arduino ; une broche numérique ne peut ni fournir son alimentation ni être reliée à VCC.
b) (1) `millis()` ; (2) `INPUT` (le capteur impose le niveau) ; (3) `HIGH` ; (4) `T_BIP` ; (5) `T_ALERTE` ; (6) `LOW` ; (7) `Etat::STANDBY`.
c) COOLDOWN ignore le capteur pendant 2 s : une même personne (ou le PIR qui reste à HIGH quelques secondes après la détection) ne redéclenche pas l'alerte en boucle ; on évite un bip permanent dès qu'il y a du passage et on laisse le capteur se rétablir.

```
// Midterm 2 (2026), Exercice 9 : alarme de magasin (blancs completes)
const int PIR = 2, LED_VERTE = 11, LED_ROUGE = 12, BUZZER = 13;
const unsigned long T_ALERTE = 3000, T_COOLDOWN = 2000, T_BIP = 300;
enum class Etat { STANDBY, ALERT, COOLDOWN };
Etat etat = Etat::STANDBY;
unsigned long tEntree = 0;

void changerEtat(Etat e, const char *nom) {
  etat = e;
  tEntree = millis();                                   // (1)
  Serial.print("State: "); Serial.println(nom);
}
void setup() {
  Serial.begin(9600);
  pinMode(PIR, INPUT);                                  // (2) le capteur impose son niveau
  pinMode(LED_VERTE, OUTPUT); pinMode(LED_ROUGE, OUTPUT); pinMode(BUZZER, OUTPUT);
```

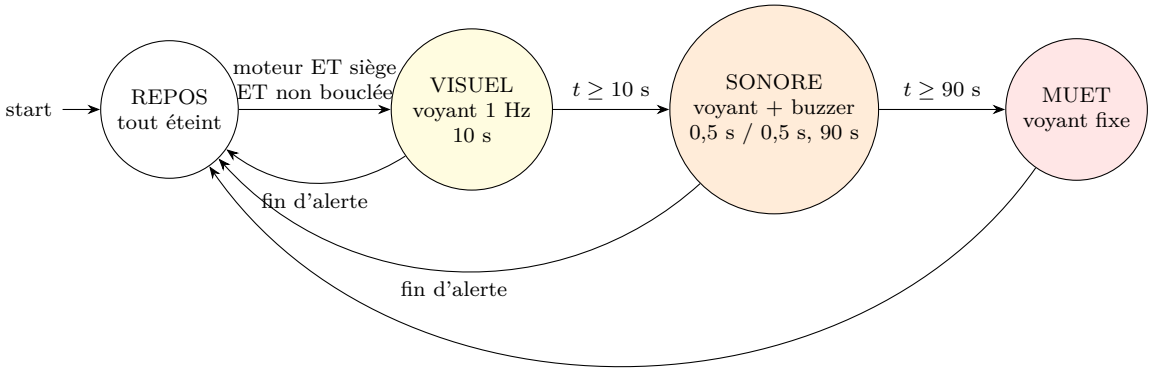
```
  changerEtat(Etat::STANDBY, "STANDBY");
}
void loop() {
  unsigned long ecoule = millis() - tEntree;
  switch (etat) {
    case Etat::STANDBY:
      digitalWrite(LED_VERTE, HIGH); digitalWrite(LED_ROUGE, LOW); digitalWrite(BUZZER, LOW);
      if (digitalRead(PIR) == HIGH)                    // (3)
        changerEtat(Etat::ALERT, "ALERT");
      break;
    case Etat::ALERT:
      digitalWrite(LED_VERTE, LOW); digitalWrite(LED_ROUGE, HIGH);
      digitalWrite(BUZZER, (ecoule / T_BIP) % 2 == 0);   // (4)
      if (ecoule >= T_ALERTE)                          // (5)
        changerEtat(Etat::COOLDOWN, "COOLDOWN");
      break;
    case Etat::COOLDOWN:
      digitalWrite(LED_ROUGE, LOW);
      digitalWrite(BUZZER, LOW);                       // (6)
      if (ecoule >= T_COOLDOWN)
        changerEtat(Etat::STANDBY, "STANDBY");         // (7)
      break;
  }
}
```

In practice. Simulateur : détection à 1 s : `State: ALERT`, LED rouge, bips de 300 ms ; `State: COOLDOWN` à 4 s, tout éteint ; `State: STANDBY` à 6 s, LED verte.

4.10 Exercice 10 : rappel de ceinture de sécurité (2,5 pts)

Réponse

a) Entrées vers GND avec pull-up interne : situation active = contact fermé = `LOW` : siège occupé `LOW`, ceinture bouclée `LOW`, moteur allumé `LOW`. Condition d'alerte : `alerte = (digitalRead(MOTEUR) == LOW) && (digitalRead(SIEGE) == LOW) && (digitalRead(CEINTURE) == HIGH)` (moteur allumé ET siège occupé ET ceinture *non* bouclée).
d) Tests : (1) *Nominal* : moteur ON, siège occupé, ceinture non bouclée pendant 2 min : voyant 1 Hz de 0 à 10 s, voyant + buzzer 0,5 s/0,5 s de 10 à 100 s, voyant fixe et silence ensuite. (2) *Bouclage pendant l'alerte sonore* : même départ, ceinture bouclée à 30 s : retour immédiat à REPOS (voyant et buzzer éteints en moins d'un tour de boucle), et aucune alerte tant qu'elle reste bouclée. (3) *Pas d'alerte sans conducteur* : moteur ON, siège libre, ceinture non bouclée pendant 30 s : rien ; puis siège occupé : l'alerte démarre à cet instant (et moteur coupé pendant l'alerte : retour à REPOS). Variante utile : rebonds du capteur de siège (passager qui bouge) ne doivent pas relancer la séquence à 0, ce que le prototype ne garantit pas (amélioration possible : hystérésis de 1 s avant REPOS).




```ino
// Midterm 2 (2026), Exercice 10 : rappel de ceinture (sketch complet)
const int SIEGE = 2, CEINTURE = 3, MOTEUR = 4, VOYANT = 8, BUZZER = 9;   // entrees vers GND, INPUT_PULLUP
const unsigned long T_VISUEL = 10000, T_SONORE = 90000UL, T_CLIGNO = 500;
enum class Etat { REPOS, VISUEL, SONORE, MUET };
Etat etat = Etat::REPOS;
unsigned long tEntree = 0;

bool siegeOccupe()     { return digitalRead(SIEGE) == LOW; }      // a) actif = LOW
bool ceintureBouclee() { return digitalRead(CEINTURE) == LOW; }
bool moteurAllume()    { return digitalRead(MOTEUR) == LOW; }
bool alerte()          { return moteurAllume() && siegeOccupe() && !ceintureBouclee(); }
void changer(Etat e) { etat = e; tEntree = millis(); }

void setup() {
  pinMode(SIEGE, INPUT_PULLUP); pinMode(CEINTURE, INPUT_PULLUP); pinMode(MOTEUR, INPUT_PULLUP);
  pinMode(VOYANT, OUTPUT); pinMode(BUZZER, OUTPUT);
}
void loop() {
  unsigned long ecoule = millis() - tEntree;
  bool phase = (ecoule / T_CLIGNO) % 2 == 0;          // 1 Hz : 500 ms ON / 500 ms OFF, sans delay
  if (etat != Etat::REPOS && !alerte()) changer(Etat::REPOS);   // retour immediat depuis tout etat d'alerte
  switch (etat) {
    case Etat::REPOS:
      digitalWrite(VOYANT, LOW); digitalWrite(BUZZER, LOW);
      if (alerte()) changer(Etat::VISUEL);
      break;
    case Etat::VISUEL:
      digitalWrite(VOYANT, phase); digitalWrite(BUZZER, LOW);
      if (ecoule >= T_VISUEL) changer(Etat::SONORE);
      break;
    case Etat::SONORE:
      digitalWrite(VOYANT, phase); digitalWrite(BUZZER, phase);
      if (ecoule >= T_SONORE) changer(Etat::MUET);
      break;
    case Etat::MUET:
      digitalWrite(VOYANT, HIGH); digitalWrite(BUZZER, LOW);
      break;
  }
}
```

In practice. Simulateur : moteur ON et siège occupé à 1 s : voyant 1 Hz ; buzzer 0,5 s/0,5 s à partir de 11 s ; voyant fixe et silence à 101 s ; ceinture bouclée à 105 s : tout éteint.

5 Appendix: the numbers and templates the papers keep asking

Summary: ATmega328P / UNO facts

8-bit AVR RISC, Harvard, 16 MHz, 5 V. Flash 32 KB (program, non-volatile), SRAM 2 KB (variables, volatile), EEPROM 1 KB (settings, non-volatile, written by the program). 14 digital pins (D0/D1 = RX/TX of `Serial`, avoid), PWM on 3, 5, 6, 9, 10, 11, 6 analog inputs A0–A5 (10-bit ADC). Per pin 40 mA absolute max, 20 mA recommended, 200 mA total. Serial receive buffer 64 bytes; `Serial.read()` returns -1 when empty; `parseInt()` skips non-digits (consuming them), waits up to 1 s, stops *before* the first non-digit after a number. `millis()` returns `unsigned long`, overflows after $\approx$ 49,7 days; in an `int` it is wrong after 32,8 s. `LED_BUILTIN` = 13. "Vérifier" compiles to a `.hex`; the core's hidden `main()` is `init(); setup(); for(;;) loop();`.

Summary: Types on the UNO

`bool`/`byte`/`char` 1 byte; `int`/`unsigned int` 2 bytes ($-32768..32767$ / 0..65535); `long`/`unsigned long`/`float`/`double` 4 bytes (`double` = `float`); `int t[10]` = 20 bytes; a `struct` of two `int` = 4 bytes (12 for three of them). `static` local = kept between calls; `const int` typed and scoped vs `#define` textual; `constexpr` = compile-time constant; `#ifdef DEBUG` includes/excludes code at compile time; `enum class` names FSM states; pass by reference `&` (or pointer) to let a function modify the caller's variable; `sizeof(tab)/sizeof(tab[0])` = number of elements.

Summary: Inputs: the three button wirings

Wiring	`pinMode`	released	pressed / closed
internal pull-up: button to GND, no resistor	`INPUT_PULLUP`	HIGH	LOW
external pull-up: 10 kΩ to 5 V, button to GND	`INPUT`	HIGH	LOW
external pull-down: 10 kΩ to GND, button to 5 V	`INPUT`	LOW	HIGH

No resistor and `INPUT`: floating input (random on the real board, "works" in simulators). Sensors with their own output (PIR OUT, IR sensor, soil sensor): `INPUT`, powered from the 5 V rail. Pull-up current when pressed: $5/R$ (0,5 mA for 10 kΩ; 50 mA and 0,25 W for 100 Ω).

Summary: LEDs and currents

$R = (5 - V_F)/I$, round *up* to the next E12 value (100, 120, 150, 180, 220, 270, 330, 390, 470…), then $I_{\text{réel}} = (5 - V_F)/R$. Chain: pin → resistor → anode (+, long leg) → cathode → GND; reversed LED = off, no damage; no resistor = LED and pin destroyed. One resistor per LED (independent branches); several LEDs on one pin: sum the currents against 40/20 mA, else transistor/MOSFET. Energy: charge per cycle $\sum I\,t$ (mAůs), I_{avg} = charge / period, autonomy = capacity (mAh) / I_{avg}.

Summary: Non-blocking templates

```
// one-shot press with 50 ms debounce (pull-up, active LOW); keep the three variables per button (
    struct, by reference)
bool appui() {
  int l = digitalRead(BTN);
  if (l != lecturePrec) { tChange = millis(); lecturePrec = l; }
  if (millis() - tChange > T_DEBOUNCE && l != etatStable) { etatStable = l; return l == LOW; }
  return false;
}
```

```
// FSM skeleton
enum class Etat { A, B };  Etat etat = Etat::A;  unsigned long tEntree = 0;
void changer(Etat e) { etat = e; tEntree = millis(); }
void loop() {
  unsigned long ecoule = millis() - tEntree;
  switch (etat) {
    case Etat::A: /* sorties de A */ if (condition) changer(Etat::B); break;
    case Etat::B: if (ecoule >= T_B) changer(Etat::A); break;
  }
}
// blink at f Hz without delay: half period T = 500/f ms
digitalWrite(LED, (ecoule / T) % 2 == 0);          // or toggle when millis() - tCligno >= T
// bip 200 ms every 5 s:  digitalWrite(BUZ, (ecoule % 5000) < 200);
```

Design checklist for the 4-point FSM: boolean expressions with the right active level; every state's outputs written in its `case`; every arrow of the diagram is one `if` with `changer()`; timeouts compare `ecoule`; "from any state" transitions tested once before the `switch`; memorise brief events in a flag; name the safe state and return to it; three test scenarios (nominal, early exit, no-trigger).